

INTERNSHIP REPORT

2026 年暑期实习总结

AI 应用开发 · Java Web 后端 · Hyperledger Fabric

实习单位 | 中电福富信息科技有限公司

实习生 | 杨企轩

时间 | 2026 年暑期

基于项目源码、测试报告与验收记录整理

实习概述

这个暑假，我围绕真实企业项目完成了三条相互关联的技术主线：一是独立搭建工厂安全隐患 AI 分析助手，将规则、知识库检索和大模型组合为可降级的本地应用；二是在既有微服务体系中完成医疗健康专区后端开发，把页面配置、四项 KPI 与新闻数据接入现有门户；三是参与 Hyperledger Fabric 星链平台的本地重建、链码修复、前后端联调与全功能验收。三项主线之外，我还完成或参与了智能对话 Agent 原型、可信数据空间智能合规审核验收材料、OpenSandbox 多运行时离线交付、Fabric 前端与接口分析、微服务配置审计、数据政策调研和企业信息安全自检。这些支线让我接触了从原型、测试、文档到部署安全的完整工程链路。

任务	核心产出	主要技术
工厂安全 AI 分析助手	多格式清单解析、三层分析、统计与历史追溯	Python、Streamlit、LangChain、DeepSeek、BGE、ChromaDB、SQLite
医疗健康专区后端	首页、KPI、新闻列表/详情接口及发布交接	Java、Spring Boot、MyBatis-Plus、MySQL、Nacos、Gateway、JUnit
Fabric 区块链平台	四组织网络、本地运行环境、链码修复、端到端验收	Hyperledger Fabric、Go、Java、Vue、Docker、WSL、Shell、Nacos、MySQL、Redis
智能对话与合规智能体	Plan-and-Execute Agent、10 条合规规则验收说明与正反样例	LangGraph、LangChain、DeepSeek、OCR、Streamlit
安全沙箱与工程支撑	多运行时离线包、风险分级调度、配置审计与可回滚交付	OpenSandbox、runc、crun、gVisor、Kata、Shell、Docker

主要任务一 工厂安全隐患 AI 分析助手

任务背景与目标

工厂安全检查清单通常包含大量自然语言描述，人工逐项查找法规、判断风险并整理整改建议，耗时且一致性不足。我实现了一个本地 Web Demo，让安全管理人员上传检查清单和企业内部安全规定，系统自动输出法规匹配、内部规定依据、原因分析、整改建议、风险优先级和统计排行。

我完成的工作

- 设计“规则匹配 → RAG 语义检索 → LLM 综合分析”的三层架构。内置规则先提供稳定基线，向量检索补充企业内部制度依据，大模型负责综合解释；任一外部能力不可用时，系统仍能降级运行。

- 建立多格式文件处理链路。Excel/CSV 通过 Pandas 与 openpyxl 本地解析，并自动匹配中英文列名；TXT、DOCX 和文本型 PDF 提取正文后，由 LLM 分段抽取结构化隐患并去重。
- 实现内部规定知识库。使用 BAAI/bge-small-zh-v1.5 生成中文向量，以 cosine 距离存入 ChromaDB；记录 embedding provider、模型、距离度量和索引版本，配置变化时自动重建。向量能力不可用时，回退到关键词/ngram 文本检索。
- 将分析流程从页面逻辑中抽离为独立服务，统一编排多文件任务、部分失败、逐条分析和结果持久化，避免 UI 与核心逻辑强耦合。
- 使用 SQLite 保存上传文件、任务、分析结果、来源行号和失败原因，以 ChromaDB 保存向量索引，实现重启后的历史追溯。
- 用 Streamlit 完成上传分析、结果展示和历史记录三类页面，提供来源文件筛选、隐患类别/部门/区域/等级/关键词统计，以及“出现次数 × 风险权重”的整改优先级排行。
- 补充两套样例数据和完整验收路径：基础清单 10 条、扩展清单 16 条，覆盖 5 种文件格式和消防、用电、机械、危化品、高处作业、叉车、检维修、个人防护等场景。

关键工程思考

这项任务让我真正理解，企业 AI 应用不能只追求“大模型能回答”。更重要的是结果可追溯、系统可降级、数据边界明确。规则层提供确定性，RAG 提供可引用的企业依据，LLM 提供自然语言归纳；三者互补，才能降低幻觉风险。对扫描版 PDF 不伪装支持、对未启用 LLM 的非结构化文件明确提示、对多文件任务记录部分失败，也让我形成了“失败必须可见、输出必须有来源”的工程习惯。

主要任务二 医疗健康专区网页后端开发

任务背景与职责边界

医疗健康专区需要接入既有门户、网关、Nacos、数据库和新闻体系。我负责的正式范围是 os-kx-common Java 后端：提供页面配置、四项 KPI、前沿动态列表与详情，并配合前端和部署人员完成接口与发布交接。菜单权限、网关白名单、目标环境数据库变更不属于单个后端模块，均在文档中明确交由对应负责人处理。

我完成的工作

- 基于 Spring Boot 存量工程新增 MedicalHealthZoneController、Service、DTO/VO、配置类、异常处理和 MyBatis Mapper，提供首页、KPI、新闻列表、新闻详情 4 个 GET 接口。
- 通过 @ConfigurationProperties(prefix = "medical-health") 管理标题、路由、Logo/Banner、统计范围、新闻数量和金额换算参数，把环境差异留给 Nacos，而不是硬编码在业务逻辑中。

- 编写只读 MyBatis 聚合查询，围绕产品、企业、订单和标签计算企业数量、数据产品数量、交易次数和交易规模；使用参数绑定、去重、状态过滤和测试数据排除，避免 SQL 拼接风险。
- 设计“医疗场景优先、同空间有效产品回退、最终无数据返回 0/NO_DATA”的稳定语义，区分“确实无数据”与“配置或 SQL 异常”。
- 复用现有新闻 Service、Entity 与 MiniO 文件能力。文件服务异常时回退到同口径数据库记录，图片按绝对地址、根相对路径、对象存储签名、配置化基础地址、默认图的顺序规范化。
- 处理本地端口、Spring context path、Gateway Rewrite 与外部 /api 前缀混淆，形成逐层 URI 验证方法，避免为适配入口层而反复改 Controller 路径。
- 对目标库导出结构进行兼容性审计，发现固定 KPI SQL 依赖的列和标签关系不完全具备，因此明确“只改 Nacos 无法解决”，并提供需 DBA 审批的最小兼容方案与只读检查脚本。
- 完成单元测试和交付材料。基线记录显示 11 项测试全部通过，同时明确单元测试不等于目标环境的 Nacos、数据库、网关、鉴权和浏览器链路已经验收。

收获与体会

这项任务让我从“写一个能跑的接口”转向“在真实系统里交付一个可维护的能力”。我学会先确认基线、包结构和现有公共能力，再做最小增量；学会把源码、配置中心、数据库、网关、Nginx、鉴权和浏览器看成一条完整请求链；也学会不把本地 200、编译成功或单元测试通过当成最终上线成功。尤其是新闻图片问题让我认识到，字符串看起来像 URL 并不代表用户端真的可访问，最终结果必须沿真实浏览器路径验证。

主要任务三 Hyperledger Fabric 区块链开发

任务背景

Fabric 星链平台包含 Portal、Gateway、Sysmgr、Info、Agent、MySQL、Redis、Nacos、Orderer、多个 Peer 与链码，既有源码、运行包、远程环境和历史材料之间存在版本差异。我的工作重点是搭建可重复的本地环境、理解并修复可信数据存证链码、梳理前后端调用链，并用真实交易完成验收。

我完成的工作

- 在 Windows + WSL Ubuntu 环境中重建本地平台，使用 Docker Compose 编排 Nacos、MySQL、Redis 和 Fabric 组件，接入 Java 服务与现有 Portal；编写 start-all.sh、smoke test 和 full-function test，使环境可重复启动和验收。
- 建立 4 组织 Fabric 网络，为 mychannel、channel2、channel3 配置 Peer 加入和链码生命周期；本地验收时 trusteddataacc 1.2.1 与 fabcar 1.0 均完成四组织批准和提交。

- 对 trusteddataacc Go 链码进行修复：解决历史记录顺序不稳定、生命周期记录未按时间排序、证书查询不兼容 recordId、evidenceData 1000 字符边界未落实等 4 类根因，对应修复 10 项失败用例。
- 为链码新增回归测试，覆盖 999/1000 个 ASCII 与中文字符边界、历史与生命周期排序、dataId/recordId 证书兼容；执行 `go test ./...`、`go vet ./...`，并从最终 ZIP 重新解压复测，避免“源码修了但交付包没更新”。
- 保持账本兼容：不修改公开方法名、参数数量、世界状态键格式和 JSON 字段，不迁移已有账本；链码升级按 Fabric v2 生命周期使用更高 version/sequence 前向发布，而不是直接替换容器或删除账本。
- 梳理 Portal 到 Fabric 的真实调用链：Vue 页面经通用 `/service/invoke|query` 接口传递 `orgId + chainCodeId + args`，Java SDK 再调用链码。对材料中不存在的专用 URL、区块高度、区块哈希和证书图片流，明确不伪造能力。
- 完成端到端验收，覆盖登录与验证码、动态菜单、区块浏览器、基础/通用/V2/生命周期存证、文件上传下载、证书、日志、合约/通道/组织/联盟/用户管理等功能；12 个业务页面可访问，真实本地交易在 4 个 Peer 上 VALID 提交。
- 对远程环境坚持只读导出与校验，保存旧环境冷备、材料包和 SHA-256 清单；敏感账号、Token 和私钥不写入报告或验收输出。

收获与体会

区块链开发让我理解了“不可篡改”背后的工程代价：链码必须确定性执行，排序不能依赖迭代器偶然方向，时间应来自交易上下文而不是各节点本地时钟；升级不能覆盖旧定义，而要经过打包、安装、各组织批准、就绪检查、提交和回查。更重要的是，分布式系统中的“请求返回成功”不一定等于交易已经最终提交，必须通过链上查询和多 Peer 状态再次确认。

次要任务与工程支撑

智能对话 Agent 原型

在 Chat_Agent_Demo 中，我先实现了基于 LangChain 与 DeepSeek 的对话 Agent，再使用 LangGraph StateGraph 扩展为 Planner、Executor、Tool、Replanner 组成的 Plan-and-Execute 工作流。原型提供知识查询、时间查询、计算、用户信息和长期记忆等 7 类工具，以 InMemorySaver 和 InMemoryStore 管理线程状态与记忆；同时加入上下文裁剪、模型与工具缓存、流式输出、工具调用展示、异常反馈和统一免责声明。这个任务让我理解了 Agent 不只是一次模型调用，而是状态、工具、路由和终止条件共同构成的可控流程。

可信数据空间智能合规审核

在可信数据文件夹中，我参与整理智能合规审核 Demo 的使用、接口联调与验收材料，并建立正反样例路径。材料覆盖主体、连接器、业务节点、数据资源和数据产品 5 类对象，对应 AI-01 至 AI-10 共 10 条规则，包括营业执照和身份证 OCR 比对、CSR 字段一致性、证明材料语义支撑、地址合理性、资源摘要匹配、个人信息脱敏以及授权委托书核验。通过把 123、4、56、7890 四组接口案例分别组织为正向和反向样例，我学习了如何让 AI 功能不仅展示“通过”，还能够稳定暴露字段不一致、未脱敏和材料不支撑等失败原因。该部分属于测试、验收与文档交付，不将其表述为底层模型或接口的独立实现。

OpenSandbox 多运行时离线交付

在 opensandbox-centos-offline-bundle 中，我围绕 CentOS 离线环境整理 runc、crun、gVisor/runsc 和 Kata Containers 的部署材料，并实现 Runtime Selector 配置管理层。系统用 runc、normal、secure、isolated 四个 profile 映射不同隔离强度，支持主机级切换、容器级运行、按 low/medium/high 风险调度、运行时验证、配置备份与回滚，以及后台容器 TTL 清理；交付包还包含校验和、安装/配置/验证/回滚脚本与故障排查文档。我在 Windows 工作区完成了代码和材料的静态交付检查，同时保留目标 Linux 主机上的验证命令，没有把静态检查写成线上部署成功。

Fabric 前端与微服务联调支撑

Fabric_Agent 中的工作补充了区块链主线的产品与交付侧能力：我梳理现有区块浏览器接口覆盖和字段缺口，区分可直接复用、需扩展和不可伪造的数据；围绕链码部署进度窗口给出计时恢复、组织状态兜底、局部网络重试和定时器清理方案，并设计对应验收用例；同时整理 SQL 测试数据的建表、播种、验证和回滚脚本以及操作手册。temp_test 中的多微服务源码与 Nacos 备份则用于核对医疗专区的配置加载顺序、Gateway 三层路径、实例状态和默认图片链路；两次不满足验收条件的最小配置尝试均按 MD5 精确回滚，没有在远端留下失败配置。

政策调研与企业安全规范

我提炼了《可信数据空间发展行动计划（2024—2028 年）》与《国家数据基础设施建设指引》，将可信管控、资源交互、价值共创以及统一目录、身份和接口等要求，与医疗健康、区块链存证和安全沙箱任务建立联系。入职和日常工作中，我还完成终端安全检查、信息安全自检、安全责任书与相关培训，进一步认识到账号凭据、客户环境和内部材料必须遵循最小暴露、可审计和可追责原则。

技术栈总结

AI 与数据处理

- Python、Streamlit、Pandas、openpyxl、python-docx、pdfplumber

- LangChain、LangGraph、DeepSeek API、Sentence Transformers、BAAI/bge-small-zh-v1.5、OCR
- RAG、Embedding、向量检索、ChromaDB、关键词/ngram 兜底
- SQLite、数据建模、多文件任务编排、统计分析

Web 后端与微服务

- Java、Spring Boot、RESTful API、DTO/VO、统一异常处理
- MyBatis/MyBatis-Plus、MySQL 5.7、参数化 SQL、聚合与回退逻辑
- Nacos 配置与服务发现、Gateway、Nginx、MinIO、Maven 私服
- JUnit/Spring Boot Test、接口契约、发布交接和环境验收

区块链与工程化

- Hyperledger Fabric v2、Peer、Orderer、MSP、Channel、链码生命周期
- Go 链码、Fabric Contract API、Fabric Java SDK
- Vue、Axios、Java 微服务网关
- Docker/Docker Compose、WSL Ubuntu、Shell、Git、SHA-256、可重复验收脚本
- OpenSandbox、runc、crun、gVisor/runsc、Kata Containers、运行时 profile 与风险分级调度

综合体会

第一，先确认事实边界，再开始编码。企业项目中，需求材料、历史 Demo、当前源码和线上运行态可能同时存在，必须明确哪个才是交付基线。第二，真正的完成标准是用户可见结果和完整链路，而不是某条命令成功。第三，稳定系统需要清晰的降级、回退和错误语义，失败不能被 mock 数据或“成功提示”掩盖。第四，安全和可维护性同样属于功能：密钥不入库、SQL 最小权限、远程环境只读、变更可回滚、产物可校验，都是工程交付的一部分。

这次实习让我把课堂中的 AI、数据库、网络、软件工程和分布式系统知识串联起来，也让我逐渐形成一套工作方法：阅读现有系统、建立证据链、做最小且可解释的修改、自动化验证、记录限制与下一步。相比单纯增加代码量，我更重要的成长是开始能够对一个真实系统的边界、风险和最终结果负责。

材料依据与说明

本总结依据工作区中的项目源码、README、复盘文档、测试报告和验收记录整理。涉及内部环境的账号、地址、Token、证书和私钥均未写入本文。医疗专区的“11 项测试通过”是本地基线结果，不代表目标环境已完成发布验收；Fabric 的功能结论对应 2026-08-20 本地重建环境；前端“仅方案”内容未被计入已完成实现。

主要依据包括：

- Factory_Safety_Analyzer_Demo/README.md 及 safety_analysis/ 核心模块
- medial_health_zone/project-retrospective/ 与 os-kx-common/ 正式源码和测试报告
- fabric-local/REBUILD_REPORT_20260820.md
- fabric-local/markdown/星链服务_全功能自检报告_20260820.md
- fabric-local/markdown/trusteddataacc 链码_10 项失败修复与部署交付说明_20260812.md
- Chat_Agent_Demo/ 中的 LangChain、LangGraph 与 Streamlit 原型源码
- 可信数据/ 中的智能合规审核说明、截图与正反接口样例
- opensandbox-centos-offline-bundle/ 中的 Runtime Selector、离线包与验证文档
- Fabric_Agent/markdown/ 中的前端方案、接口审计、测试数据与操作手册
- temp_test/nacos-backup/nacos-gateway-config-summary.md
- 两份国家数据局材料核心要点提炼.md 与 实习/ 中的信息安全材料